

Assignment 7: Sensor Models and Attitude Determination

1 Introduction

In this assignment the student will learn how to implement models for attitude sensors and how to use them for attitude determination. This will be done by extending `orbit_lib.py` and `sat_lib.py` with functionality for simulating sensor and determining the satellite's attitude based on sensor measurements.

After completing this assignment the student will learn:

- How to implement sensor models and attitude determination algorithms.

2 Preliminaries

2.1 Earth Model

We will model the Earth as a spheroid with parameters based on WGS-84

- Equatorial radius: $R_E = 6378.137$ km
- Flattening: $f = \frac{1}{298.257223563}$
- Second Zonal Harmonic: $J_2 = 0.001082629821313$
- Rotational velocity: $\omega_E = 7.292115 \cdot 10^{-5}$ rad/s
- Gravitational parameter: $\mu = 398600.4418$ km³/s²

2.2 Simulating Noise

Often we need to be able to simulate noise with certain statistical properties. A normal distribution with mean μ and variance σ^2 (remember that the standard deviation is the square-root of the variance σ) can be sampled by shifting and scaling samples from the standard normal distribution ($\mu = 0$ and $\sigma^2 = 1$)

$$x = \sigma \cdot \eta + \mu$$

where η is sampled from the standard normal distribution. In python generating samples from a normal distribution is easily done using numpy

```
1 np.random.normal() #mean = 0, variance = 1
2 np.random.normal(loc=10, scale=2) #mean = 10, variance =
  scale^2
```

If a random variable x is normally distributed (gaussian) with mean μ and variance σ^2 we denote it $x \sim N(\mu, \sigma^2)$.

We can define a random vector $\mathbf{v} = [v_1 \ v_2 \ \dots \ v_n]^\top$ as n with mean $\boldsymbol{\mu} = [\mu_1 \ \mu_2 \ \dots \ \mu_n]^\top$ and a covariance $\mathbf{Q}$

$$\mathbf{Q} = \begin{bmatrix} \sigma_1^2 & \sigma_{12} & \dots & \sigma_{1n} \\ \sigma_{12} & \sigma_2^2 & \dots & \vdots \\ \vdots & \vdots & \ddots & \vdots \\ \sigma_{1n} & \dots & \dots & \sigma_n^2 \end{bmatrix}$$

which is positive definite and symmetric with size $N \times N$. The off-diagonal elements signify that v_i and v_j are correlated. If v_i, v_j are uncorrelated for $i \neq j$ then $\sigma_{ij} = 0$ such that $\mathbf{Q}$ is diagonal. If v_i, v_j are uncorrelated we can sample each element separately $v_j \sim N(\mu_i, \sigma_i^2)$. If the elements are correlated we can perform a spectral decomposition of $\mathbf{Q}$

$$\mathbf{Q} = \mathbf{P} \mathbf{D} \mathbf{P}^\top$$

and a sample v_i can be generated as

$$v_i = \sqrt{\lambda_i} \cdot \eta + \mu_i$$

where λ_i is the i th diagonal element of $\mathbf{D}$. In python we can use numpy which does all this for us. To generate a sample from $N(\boldsymbol{\mu}, \mathbf{Q})$ we simply do

```
1 v = np.random.multivariate_normal(mu,Q)
```

It is common to assume that sensor noise is gaussian and white where white means that the noise at any two instances of time i, j is uncorrelated. Sometimes we want to sample from a uniform distribution $U(a, b)$, which can be done using the standard uniform distribution $U(0, 1)$

$$x = (b - a)u + a$$

$u \sim U(0, 1)$. With numpy we simply do

```
1 x = np.random.uniform(low=a,high=b)
```

2.3 Sensor Models

2.3.1 Rate Gyros

A three-axis rate gyro can be modeled as

$$\begin{aligned} \mathbf{g}_{is}^s &= \mathbf{q}_{bs}^* \otimes \boldsymbol{\omega}_{ib}^b \otimes \mathbf{q}_{bs} + \mathbf{b}_g + \boldsymbol{\eta}_g \\ \dot{\mathbf{b}}_\omega &= \boldsymbol{\eta}_{b_\omega} \end{aligned}$$

where $\boldsymbol{\eta}_g \sim N(0, \sigma_g^2 \mathbf{I})$ and $\boldsymbol{\eta}_{b_\omega} \sim N(0, \sigma_{b_\omega}^2 \mathbf{I})$ are zero mean, gaussian white noise. If the sensor's axes are aligned with the satellite's body frame then $\mathbf{q}_{bs} = \mathbf{q}_I$.

2.3.2 Sun Sensors

A model for a coarse sun sensor is

$$I = I_{\max} \cdot (1 + k_I) \mathbf{n}^{b\top} \hat{\mathbf{s}}_t^b + \eta_I$$

with $\mathbf{n}^b$ being the sun sensor's viewing direction, $\hat{\mathbf{s}}_t^b$ is the true unit vector pointing from the sensor to the sun, k_I is a scaling factor and $\eta_I \sim N(0, \sigma_I^2)$. This model requires knowing the true sun direction $\hat{\mathbf{s}}_t^i$ which requires a sun model of which there exist several. A simple one that is good enough for most applications is

$$\begin{aligned} T &= (JD - 2451545.0)/(36525.0) \\ \lambda_M &= 280.46^\circ + 36000.771 \cdot T \\ M &= 357.5291092^\circ + 35999.05034 \cdot T \\ \varepsilon &= 23.439291^\circ - 0.0130042 \cdot T \end{aligned}$$

Reduce λ_M and M to the range 0° to 360° and convert M to radians, then

$$\lambda_e = \lambda_M + 1.914666471^\circ \sin(M) + 0.019994643 \sin(2M)$$

convert λ_e to radians and finally

$$\begin{aligned} r &= \text{AU} \cdot (1.000140612 - 0.016708617 \cos(M) - 0.000139589 \cos(2M)) \\ \mathbf{s}^i &= r \begin{bmatrix} \cos(\lambda_e) \\ \cos(\varepsilon) \sin(\lambda_e) \\ \sin(\varepsilon) \sin(\lambda_e) \end{bmatrix} \end{aligned}$$

where $AU = 149597870$ km. This gives the true position of the sun relative to the Earth in ECI frame and $\hat{\mathbf{s}}_t^b$ can be found as

$$\hat{\mathbf{s}}_t^b = \frac{\mathbf{q}_{ib}^* \otimes (\mathbf{s}^i - \mathbf{r}^i) \otimes \mathbf{q}_{ib} + \mathbf{p}_I^b}{\|\mathbf{q}_{ib}^* \otimes (\mathbf{s}^i - \mathbf{r}^i) \otimes \mathbf{q}_{ib} + \mathbf{p}_I^b\|}$$

where $\mathbf{p}_I^b$ is the position of the sun sensor relative to the position of the satellite's center of mass i.e. $\mathbf{r}^i$. Typically we can approximate $\hat{\mathbf{s}}_t^b$ as

$$\hat{\mathbf{s}}_t^b \approx \mathbf{q}_{ib}^* \otimes \frac{\mathbf{s}^i}{\|\mathbf{s}^i\|} \otimes \mathbf{q}_{ib}$$

Sun sensor measurements are not available during eclipses and we therefore need to be able to determine this. A simple model that assumes a spherical Earth is

$$\begin{aligned} C &= \frac{R_E \|\mathbf{s}^i\|}{R_s - R_E} \\ \rho_s &= \arcsin \frac{R_s}{\|\mathbf{s}^i - \mathbf{r}^i\|} \\ \rho_E &= \arcsin \frac{R_E}{\|\mathbf{r}^i\|} \\ \theta &= \arccos \frac{(\mathbf{s}^i - \mathbf{r}^i)^\top (-\mathbf{r}^i)}{\|\mathbf{s}^i - \mathbf{r}^i\| \|\mathbf{r}^i\|} \end{aligned}$$

such that the satellite is in Earth shadow when $\|\mathbf{s}^i\| < \|\mathbf{s}^i - \mathbf{r}^i\| < \|\mathbf{s}^i\| + C$ and $\theta < \rho_E - \rho_s$. The measured sun vector can then be found from combining measurements from several sun sensors.

A 2-axis fine sun sensor with a field of view α and pointing direction along the sensor frame's z-axis can be modelled as

$$\begin{aligned} \hat{\mathbf{s}}_t^s &= \mathbf{q}_{bs}^* \otimes \hat{\mathbf{s}}_t^b \otimes \mathbf{q}_{bs} = \begin{bmatrix} x \\ y \\ z \end{bmatrix} \\ \hat{\mathbf{s}}_m^s &= \begin{cases} \hat{\mathbf{s}}_t^s + \boldsymbol{\eta}_s & \text{If } z > 0 \text{ and } \arctan\left(\frac{\sqrt{x^2+y^2}}{z}\right) < \frac{\alpha}{2} \\ \mathbf{0} & \text{otherwise} \end{cases} \end{aligned}$$

$$\boldsymbol{\eta}_s \sim N(0, \sigma_s^2 \mathbf{I}).$$

2.3.3 Magnetometer

A model for a magnetometer is

$$\mathbf{B}_m^s = \mathbf{M}_B ((\mathbf{q}_{ib} \otimes \mathbf{q}_{bs})^* \otimes \mathbf{B}_E^i \otimes (\mathbf{q}_{ib} \otimes \mathbf{q}_{bs})) + \mathbf{b}_B + \boldsymbol{\eta}_B$$

where $\mathbf{M}_B$ are the soft iron disturbances and $\mathbf{b}_B$ are the hard iron disturbances, $\mathbf{B}_E^i$ is Earth's magnetic field vector at $\mathbf{r}^i$, while $\boldsymbol{\eta}_B \sim N(0, \sigma_B^2)$. Typically using magnetometer measurements require preprocessing where a calibrated measurement $\mathbf{B}_c^s$ is obtained using an estimate of $\mathbf{M}_B$ and $\mathbf{b}_B$ such that

$$\mathbf{B}_c^s = \hat{\mathbf{M}}_B^{-1} (\mathbf{B}_m^s - \hat{\mathbf{b}}_B)$$

If we assume that the we receive the measurements after preprocessing (or that there are no hard iron or soft iron disturbances) then we can simplify the model to

$$\mathbf{B}_m^s = \mathbf{q}_{is}^* \otimes \mathbf{B}_E^i \otimes \mathbf{q}_{is} + \boldsymbol{\eta}_B$$

As with the Sun sensor's we need a mathematical model of the Earth's magnetic field. We'll use a dipole model where

$$\mathbf{B}_E^i = \frac{-3\mathbf{r}^{i\top}\mathbf{m}^i + \|\mathbf{r}^i\|^2\mathbf{m}^i}{\|\mathbf{r}^i\|^5}$$

where $\mathbf{m}^i$ is Earth's magnetic moment which we will define to point in the direction towards the magnetic north pole

$$\mathbf{m}^i = m\mathbf{R}_E^i \begin{bmatrix} \cos \phi \cos \lambda \\ \sin \phi \cos \lambda \\ \sin \lambda \end{bmatrix}$$

with $m = 7.767 \cdot 10^6 \text{T}\cdot\text{km}^3$, $\phi = -72.76^\circ$ and λ are the geocentric longitude and latitude respectively. Given the geodetic latitude $\lambda' = 9.21^\circ$ the geocentric latitude can be found as $\lambda = \arctan(1 - f)^2 \tan \lambda'$.

2.3.4 Star Tracker

A a simplified model of a star tracker would be

$$\hat{\mathbf{q}}_{is} = \mathbf{q}_{ib} \otimes \mathbf{q}_{bs} \otimes \mathbf{q}_\eta$$

where $\mathbf{q}_\eta$ is the noise. If we were to be rigorous we would use the Bingham distribution to sample $\mathbf{q}_\eta$, but we instead if we assume the noise is small then

$$\mathbf{q}_\eta \approx \begin{bmatrix} 1 \\ \frac{1}{2}\theta_\eta \mathbf{u}_\eta \end{bmatrix}$$

so we will instead sample an axis-angle $\theta_\eta \mathbf{u}_\eta$ and then generate $\mathbf{q}_\eta$. If we let $\theta_\eta \sim N(0, \sigma_\eta^2)$, then we just need to generate a random unit vector which can be done as

$$\begin{aligned} \theta &= \arccos(1 - 2x) \\ \phi &= 2\pi y \\ \mathbf{u} &= \begin{bmatrix} \cos \phi \sin \theta \\ \sin \phi \sin \theta \\ \cos \theta \end{bmatrix} \end{aligned}$$

with $x \sim U(0, 1)$ and $y \sim U(0, 1)$, which uniformly samples the unit sphere.

2.4 Attitude Determination

2.4.1 TRIAD

Given two sets of vectors $\mathbf{a}^o, \mathbf{a}^b$ and $\mathbf{b}^o, \mathbf{b}^b$ then the attitude can be reconstructed using the TRIAD algorithm

TRIAD: If $\mathbf{a}^b$ is less noisy than $\mathbf{b}^b$, then $\mathbf{U}^o = \mathbf{a}^o$, $\mathbf{U}^b = \mathbf{a}^b$, $\mathbf{V}^o = \mathbf{b}^o$, $\mathbf{V}^b = \mathbf{b}^b$ otherwise $\mathbf{U}^o = \mathbf{b}^o$, $\mathbf{U}^b = \mathbf{b}^b$, $\mathbf{V}^o = \mathbf{a}^o$, $\mathbf{V}^b = \mathbf{a}^b$

$$\begin{aligned} \mathbf{t}_1^o &= \frac{\mathbf{U}^o}{\|\mathbf{U}^o\|} & \mathbf{t}_1^b &= \frac{\mathbf{U}^b}{\|\mathbf{U}^b\|} \\ \mathbf{t}_2^o &= \frac{\mathbf{t}_1^o \times \mathbf{V}^o}{\|\mathbf{t}_1^o \times \mathbf{V}^o\|} & \mathbf{t}_2^b &= \frac{\mathbf{t}_1^b \times \mathbf{V}^b}{\|\mathbf{t}_1^b \times \mathbf{V}^b\|} \\ \mathbf{t}_3^o &= \mathbf{t}_2^o \times \mathbf{t}_1^o & \mathbf{t}_3^b &= \mathbf{t}_2^b \times \mathbf{t}_1^b \end{aligned}$$

The estimated attitude is then

$$\hat{\mathbf{R}}_b^o = [\mathbf{t}_1^o \quad \mathbf{t}_2^o \quad \mathbf{t}_3^o] [\mathbf{t}_1^b \quad \mathbf{t}_2^b \quad \mathbf{t}_3^b]^\top$$

which can be converted to a quaternion using Shepperd's algorithm.

2.4.2 Davenport's q method

Given N sets of unit vectors $\mathbf{u}_i^o$, $\mathbf{u}_i^b$ and weights a_i , $i = 1, 2, \dots, N$, the attitude can be reconstructed using Davenport's q method.

Davenport: Compute λ_0 , $\mathbf{B}$ and $\mathbf{z}$

$$\lambda_0 = \sum_{i=1}^N a_i, \quad \mathbf{B} = \sum_{i=1}^N a_i \mathbf{u}_i^b \cdot (\mathbf{u}_i^o)^\top, \quad \mathbf{z} = \sum_{i=1}^N a_i \mathbf{u}_i^b \times \mathbf{u}_i^o$$

where $\mathbf{u}_i^b \cdot (\mathbf{u}_i^o)^\top$ is the outer product and use these to construct the $\mathbf{K}$ matrix

$$\mathbf{K} = \begin{bmatrix} \text{tr}(\mathbf{B}) & \mathbf{z}^\top \\ \mathbf{z} & \mathbf{B} + \mathbf{B}^\top - \text{tr}(\mathbf{B}) \mathbf{I} \end{bmatrix}.$$

Now the eigenvalues and eigenvectors can be computed using numpy

```
1 import numpy as np
2 evals, evecs = np.linalg.eig(K)
```

and the eigenvector corresponding to the largest eigenvalue can then be normalized to form the attitude estimate $\hat{\mathbf{q}}_{ob}$. If we want to weight the sets of vectors evenly we can set $a_i = \frac{1}{N}$.

3 Assignment 7: Attitude Determination

We define the sensor classes to have a common interface, mainly three functions:

$\text{init}(\mathbf{q}_{bs}, \mathbf{p}^b, \mu, \mathbf{Q}, \mathbf{z}_0, \text{params})$: orientation, position, mean, covariance, initial output, optional parameters used by some sensor class that have parameters specific to them.

$\text{update}(t, t_{\text{step}}, \mathbf{q}_{ib}, \boldsymbol{\omega}_{ib}^b, \mathbf{r}^i, \mathbf{v}^i)$: updates the sensors internal state and output.

output: returns current output in either sensor frame or body frame.

An example of a Star Tracker class would then be

```

1 class star_tracker:
2     def __init__(q_bs=su.Quaternion(), p_b=np.array([0,0,0]), z0
      =0, mu=0, Q=0, params=None):
3         self.q_bs = q_bs
4         self.p = p_b
5         self.mu = mu
6         self.Q = Q
7         self.z = z0
8     def update(t, t_step, q_ib, w_b_ib, r_i, v_i):
9         theta = np.random.rand(mu, Q)
10        x, y = np.random.rand(2)
11        theta = np.arccos(1-2*x)
12        phi = 2*np.pi*y
13        u = np.array([np.cos(phi)*np.sin(theta), np.sin(phi)*np.
      sin(theta), np.cos(theta)])
14        q_e = su.Quaternion(theta, u)
15        self.z = q_ib@self.q_bs@q_e
16    def output(body_frame=False):
17        if body_frame:
18            return self.q_bs.rotate(self.z)
19        else:
20            return self.z

```

Since all sensors have the same interface they can be put in a list which makes it easy to update them and gather sensor measurements

```

1 measurements=[]
2 for sensor in self.sensors:
3     sensor.update(t, t_step, q_ib, w_b_ib, r_i, v_i)
4     measurements.append(sensor.output(body_frame=True))

```

We define the attitude determination classes to have a common interface, mainly the functions

$\text{init}(\text{params})$: optional parameters.

$\text{estimate_attitude}(\mathbf{M}_B, \mathbf{M}_A)$: returns estimated coordinate transformation from A to B

as a quaternion the estimated coordinate transformation would be $\hat{\mathbf{q}}_{BA}$. $\mathbf{M}_B$ is a list of vectors in frame B, and $\mathbf{M}_A$ is a list of vectors in frame A. The corresponding indices in $\mathbf{M}_B$ and $\mathbf{M}_A$ should be related, i.e. $\mathbf{M}_B[i] = \mathbf{q}_{BA} \otimes \mathbf{M}_A[i] \otimes \mathbf{q}_{BA}^*$.

Assignment 7

1. Extend Orbit_lib.py with the following functions

$$\mathbf{B}^i = \text{magnetic_field_dipole}(\mathbf{r}_i, \text{JD})$$

$$\mathbf{s}^i = \text{sun_vector}(\text{JD})$$

2. Extend sat_lib.py with the following sensor classes

gyro

magnetometer

fine_sun_sensor

3. Extend sat_lib.py with the following attitude determination classes

TRIAD

Davenport

4. Extend the satellite class to include one gyro sensor, one magnetometer and six fine sun sensors.

5. Extend the ADCS_PD to include attitude determination:

- Change the init function to also take an attitude estimator as an argument.
- Change the update function to have magnetometer and fine sun sensor measurements instead of $\mathbf{q}_{ib}$ as argument. Use the measurements together with the sun vector model and Earth's magnetic field model to obtain $\hat{\mathbf{q}}_{ob}$. Also replace $\boldsymbol{\omega}_{ib}^b$ with the gyro measurement.

6. Simulate Assignment 6 and use the sensor parameters given in Appendix A. We will make the reasonable assumption that the Earth is transparent (no eclipses). Try some other noise parameters and see how it affects the attitude performance.

Table 1: Magnetometer and Rate Gyro parameters.

Sensor	Parameter	Value
Gyro:	μ	
	σ_g^2	0.1
	p^b	$[0 \ 0 \ 0]^\top$
	q_{bs}	$[1 \ 0 \ 0 \ 0]^\top$
	b_g	$\mathbf{0}$
	$\sigma_{b_g}^2$	0
Magnetometer	μ	
	σ_B^2	0.4
	p^b	$[0 \ 0 \ 0]^\top$
	q_{bs}	$[1 \ 0 \ 0 \ 0]^\top$
	b_B	$\mathbf{0}$
	M_B	I

A Sensor parameters

The satellite has one rate gyro, one magnetometer and six fine sun sensors. The parameters of each sensor is given in Table 1 and Table 2

Table 2: Fine Sun Sensor parameters.

Sensor	Parameter	Value
Fine Sun Sensor x	$\boldsymbol{\mu}$	
	$\boldsymbol{\sigma}_s^2$	0.2
	$\boldsymbol{p}^b$	$[0.1 \ 0 \ 0]^\top$
	$\boldsymbol{q}_{bs}$	$[\cos \pi/4 \ 0 \ \sin \pi/4 \ 0]^\top$
	α	π
Fine Sun Sensor -x	$\boldsymbol{\mu}$	
	$\boldsymbol{\sigma}_s^2$	0.2
	$\boldsymbol{p}^b$	$[-0.1 \ 0 \ 0]^\top$
	$\boldsymbol{q}_{bs}$	$[\cos \pi/4 \ 0 \ -\sin \pi/4 \ 0]^\top$
	α	π
Fine Sun Sensor y	$\boldsymbol{\mu}$	
	$\boldsymbol{\sigma}_s^2$	0.2
	$\boldsymbol{p}^b$	$[0 \ 0.1 \ 0]^\top$
	$\boldsymbol{q}_{bs}$	$[\cos \pi/4 \ -\sin \pi/4 \ 0 \ 0]^\top$
	α	π
Fine Sun Sensor -y	$\boldsymbol{\mu}$	
	$\boldsymbol{\sigma}_s^2$	0.2
	$\boldsymbol{p}^b$	$[0 \ -0.1 \ 0]^\top$
	$\boldsymbol{q}_{bs}$	$[\cos \pi/4 \ \sin \pi/4 \ 0 \ 0]^\top$
	α	π
Fine Sun Sensor z	$\boldsymbol{\mu}$	
	$\boldsymbol{\sigma}_s^2$	0.2
	$\boldsymbol{p}^b$	$[0 \ 0 \ 0.1]^\top$
	$\boldsymbol{q}_{bs}$	$[1 \ 0 \ 0 \ 0]^\top$
	α	π
Fine Sun Sensor -z	$\boldsymbol{\mu}$	
	$\boldsymbol{\sigma}_s^2$	0.2
	$\boldsymbol{p}^b$	$[0 \ 0 \ -0.1]^\top$
	$\boldsymbol{q}_{bs}$	$[0 \ 1 \ 0 \ 0]^\top$
	α	π